

Class 6: R functions

Chaewoo Chun (A18548909)

All functions in R have at least 3 things:

-A **name**, we pick this and use it to call the function. - Input **arguments**, there can be multiple comma separated inputs to the function - The **body**, lines of R code that do the work of the function.

Our first wee function:

```
add <- function(x,y=1) {  
  x+y  
}
```

Let's test out function

```
add(c(1,2,3), y=10)
```

```
[1] 11 12 13
```

```
add(10,10)
```

```
[1] 20
```

A second function

Let's try something more interesting. Make a sequence generation tool.

The 'sample()' function could be useful here.

```
sample(1:10, size = 3)
```

```
[1] 2 7 9
```

change this to work with nucleotide A C G and T and return 3 of them

```
n <- c ("A","C","G","T")
sample(n,size= 20,replace= TRUE)
```

```
[1] "G" "T" "G" "T" "G" "C" "A" "A" "G" "G" "A" "C" "T" "T" "T" "T" "A" "G" "T"
[20] "A"
```

Turn this snippet into a function that returns a user specified length DNA sequence... Let's call it 'generate_dna()...

```
generate_dna <- function(len=10, fasta=FALSE) {
  n <- c ("A","C","G","T")
  v <-sample(n,size= len,replace= TRUE)

  # Make a single element vector
  s<- paste(v, collapse="")

  cat("Well done you!")
  if(fasta){
    return(s)
  } else{return(v)}
}
```

```
generate_dna(5)
```

Well done you!

```
[1] "A" "C" "A" "C" "G"
```

```
s <- generate_dna(15)
```

Well done you!

```
s
```

```
[1] "G" "G" "C" "A" "T" "C" "G" "G" "G" "C" "C" "T" "G" "C" "T"
```

I want the option to return a single element character vector with my sequence all together like this: "GGAGTAC"

```
generate_dna(10,fasta= FALSE)
```

Well done you!

```
[1] "G" "A" "C" "C" "C" "A" "G" "C" "A" "G"
```

```
generate_dna(10,fasta= TRUE)
```

Well done you!

```
[1] "CGAGATCGCC"
```

A more advanced example

Make a third function that generates protein sequence of a user specified length and format.

```
generate_protein <- function(size = 15, fasta = TRUE) {  
  aa <- c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I",  
         "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")  
  seq <- sample(aa, size = size, replace = TRUE)  
  
  if (fasta) {  
    return(paste(seq, collapse = ""))  
  } else {  
    return(seq)  
  }  
}
```

try this out,,,

```
generate_protein(10)
```

```
[1] "KGDKYKPFNW"
```

```
generate_protein(10 , fasta= FALSE)
```

```
[1] "G" "Y" "T" "G" "K" "S" "G" "E" "E" "D"
```

Q. Generate random protein sequences between lengths 5 and 12 amino acids.

```
generate_protein(5)
```

```
[1] "CWVQM"
```

```
generate_protein(12)
```

```
[1] "MCLMYAFCAICW"
```

One approach is to do this by brute force calling our function for each length 5 to 12.

Another approach is to write a 'for()' loop to iterate over the input values 5 to 12

A very useful third R specific approach is to use the 'sapply()' function.

```
seq_lengths <- 5:12
for ( i in seq_lengths) {
  cat(">",i,"\n")
  cat(generate_protein(i))
  cat("\n")
}
```

```
> 5
PFSFI
> 6
YSTYYS
> 7
YKQDACG
> 8
MNSWSNFH
> 9
FENCQYIRC
> 10
LSTLQYHCYT
> 11
VEPLNYCFYQP
> 12
RTHTVMGKKARM
```

```
sapply(5:12, generate_protein)
```

```
[1] "WAGNL"      "PANWWD"      "ERPRIPG"      "VWFCYDPL"      "RHKPPHEVQ"  
[6] "ETILKLHDAI" "PEWQVRLDSQM" "EQNHSGTQSRTY"
```

Key- Point: Writing functions in R is doable but not the easiest thing. Starting with a working snippet of code and then using LLM tools to improve generalize your function code is a productive approach.